

Instituto Tecnológico de San Juan del Río



Temas de ciberseguridad

Reconstrucción del Código Fuente con Ghidra

P R E S E N T A:

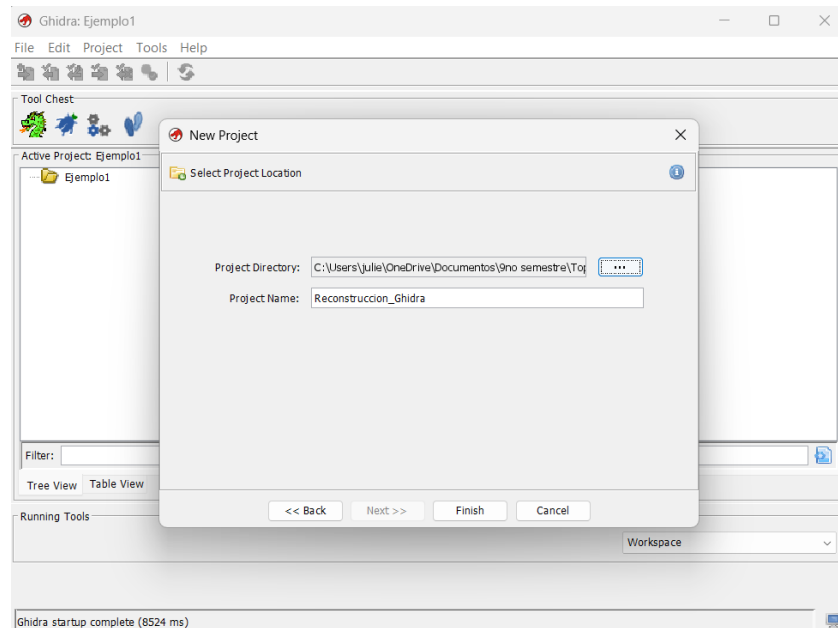
Angelica Enriquez Yañez 22590325
Daniela Dorantes Callejas 22590323
Julieta Becerril Ruiz 22590320

Ingeniería en sistemas Computacionales

PERIODO [Agosto - Diciembre]

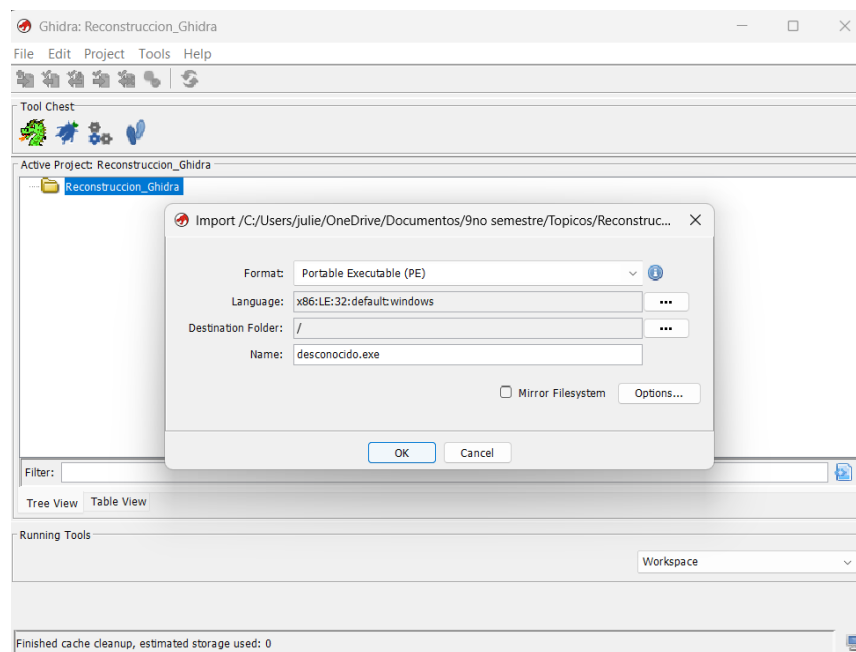
1. Crear el proyecto en Ghidra

En Ghidra se crea un proyecto nuevo, se le asigno el nombre de "Reconstruccion_Ghidra":

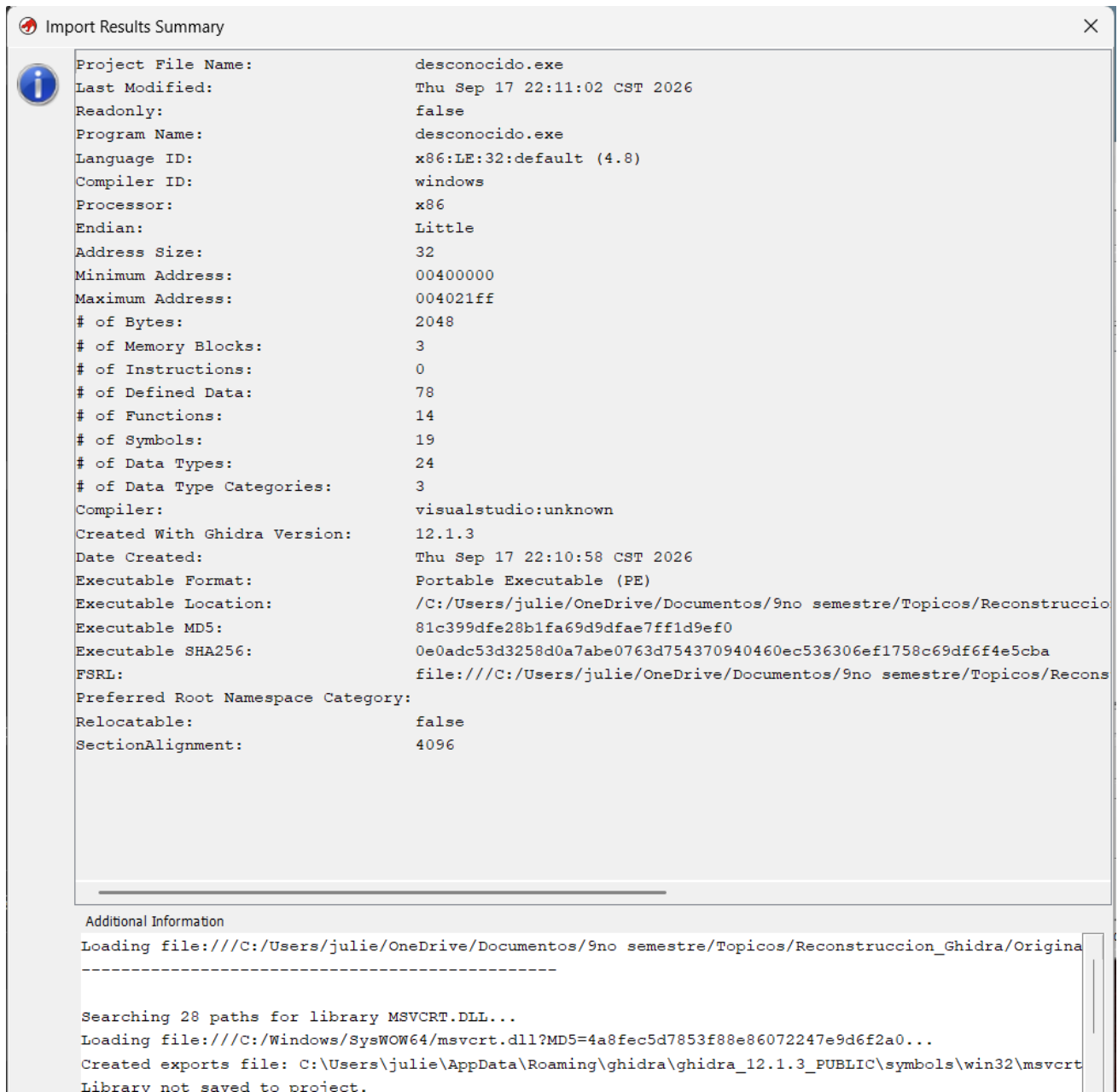


2. Importar el ejecutable

Con File → Import File importe el desconocido.exe. Ghidra detecto que era un "Portable Executable" (formato de Windows) de 32 bits, así que se deja esa configuración:

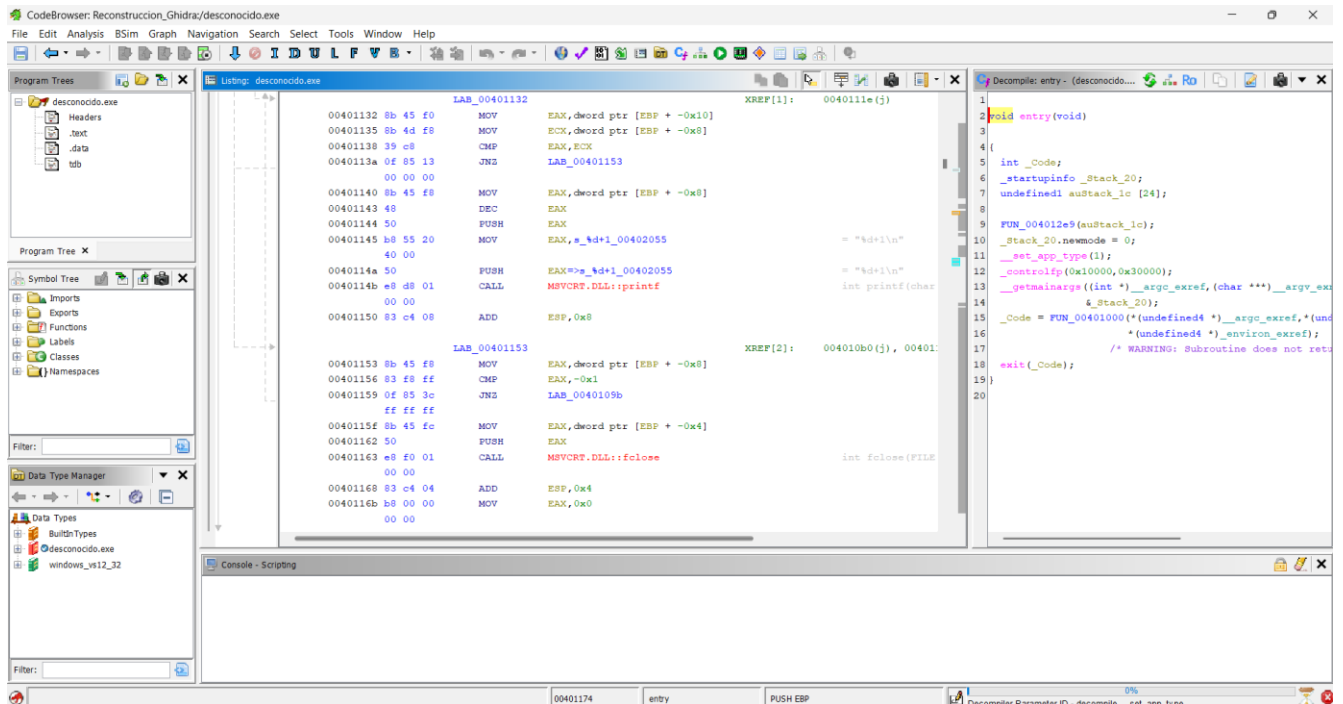


Al terminar me mostró un resumen: tamaño del archivo, cuántas funciones encontró, los hashes, etc. Aquí se confirma que el MD5 coincidía con el anterior:



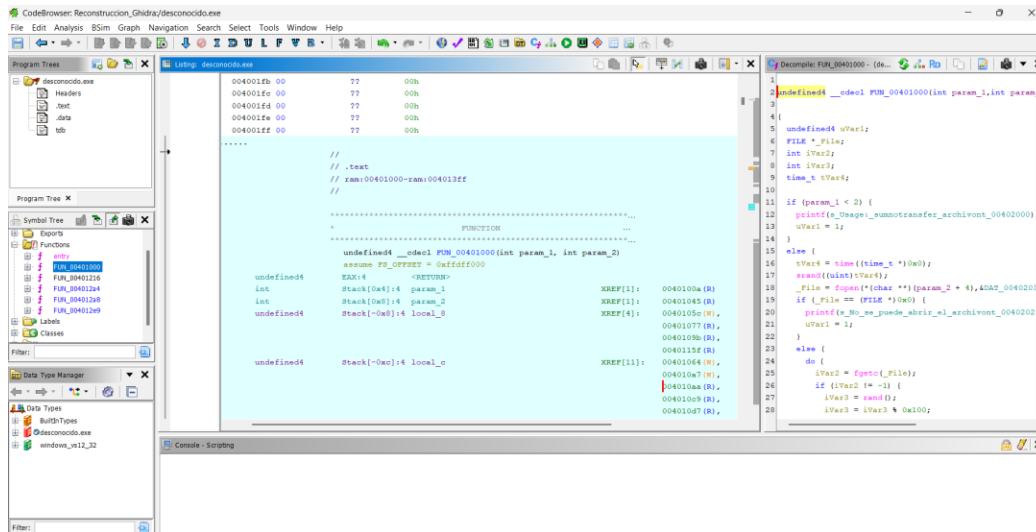
3. Encontrar cuál función es el "main"

Al abrir el binario en Ghidra, la primera función que aparece es entry, pero esta no es el main del código original: es un bloque que el compilador (TCC) agrega automáticamente para preparar el programa antes de ejecutarlo. Para encontrar el main real, se identificó qué función llama entry justo antes de invocar exit. Esto funciona porque en C, el código de arranque siempre recibe el valor que retorna main y se lo pasa a exit — por lo que esa función intermedia es, con certeza, la función principal del programa.



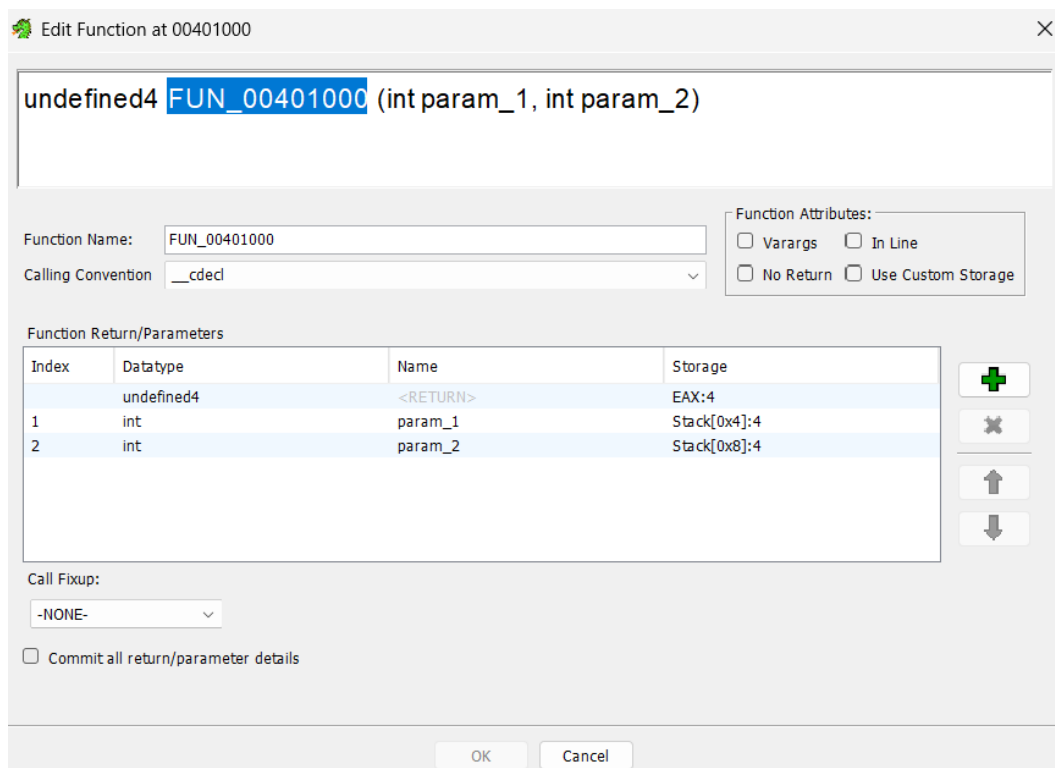
4. Abrir esa función y ver el pseudocódigo

En la lista de funciones, FUN_00401000 resultó ser la única que no pertenece al compilador; el resto corresponde al runtime interno de TCC. Al abrirla, Ghidra mostró su pseudocódigo —una traducción aproximada a lenguaje C— pero con nombres genéricos (param_1, local_8, undefined4, etc.), ya que el decompilador no puede inferir el significado real de cada variable sin ayuda del analista.

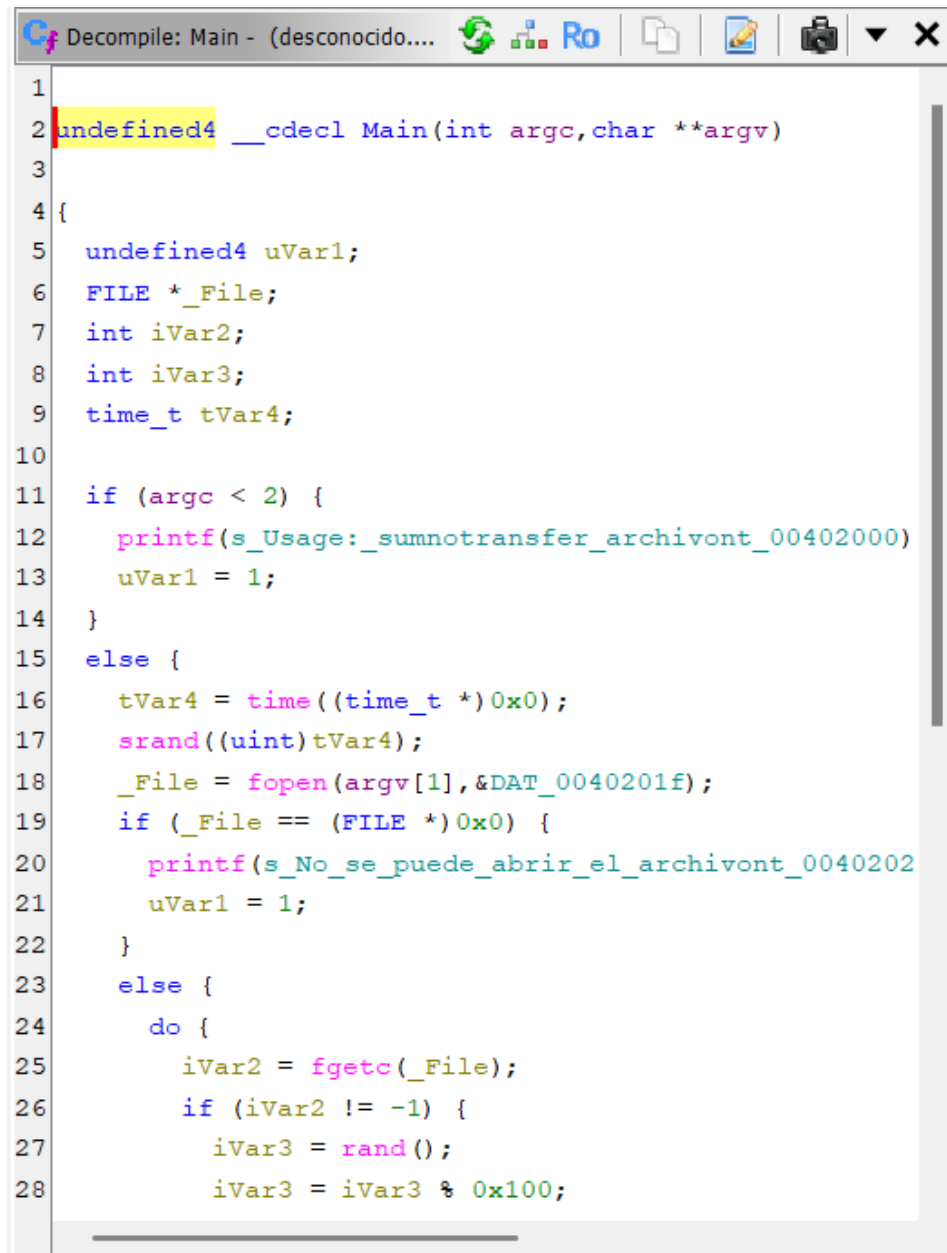


5. Arreglar el nombre y el tipo de la función

Ghidra tiene la opción Edit Function Signature para modificar cómo se representa una función. Inicialmente, la firma se mostraba como undefined4 FUN_00401000(int param_1, int param_2), lo que evidencia que el decompilador aún no reconocía esta función como un main estándar de C.

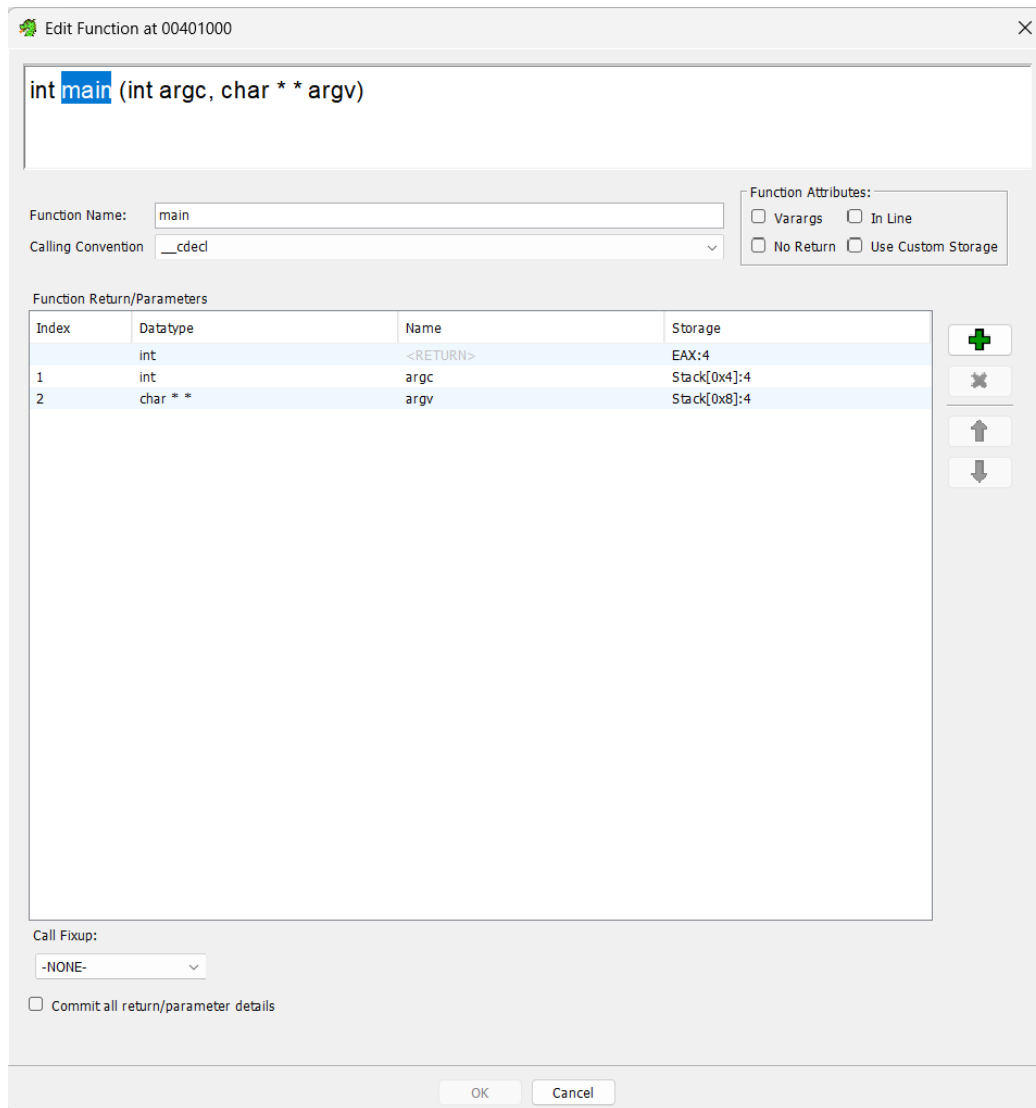


La firma se corrigió manualmente: se asignó el nombre main, el tipo de retorno int, y se renombraron los parámetros como argc y argv convención estándar en C para los argumentos que un programa recibe desde la línea de comandos.



```
1
2 undefined4 __cdecl Main(int argc, char **argv)
3
4 {
5     undefined4 uVar1;
6     FILE *_File;
7     int iVar2;
8     int iVar3;
9     time_t tVar4;
10
11     if (argc < 2) {
12         printf(s_Usage:_sumnottransfer_archivont_00402000)
13         uVar1 = 1;
14     }
15     else {
16         tVar4 = time((time_t *)0x0);
17         srand((uint)tVar4);
18         _File = fopen(argv[1], &DAT_0040201f);
19         if (_File == (FILE *)0x0) {
20             printf(s_No_se_puede_abrir_el_archivont_0040202
21             uVar1 = 1;
22         }
23         else {
24             do {
25                 iVar2 = fgetc(_File);
26                 if (iVar2 != -1) {
27                     iVar3 = rand();
28                     iVar3 = iVar3 % 0x100;
```

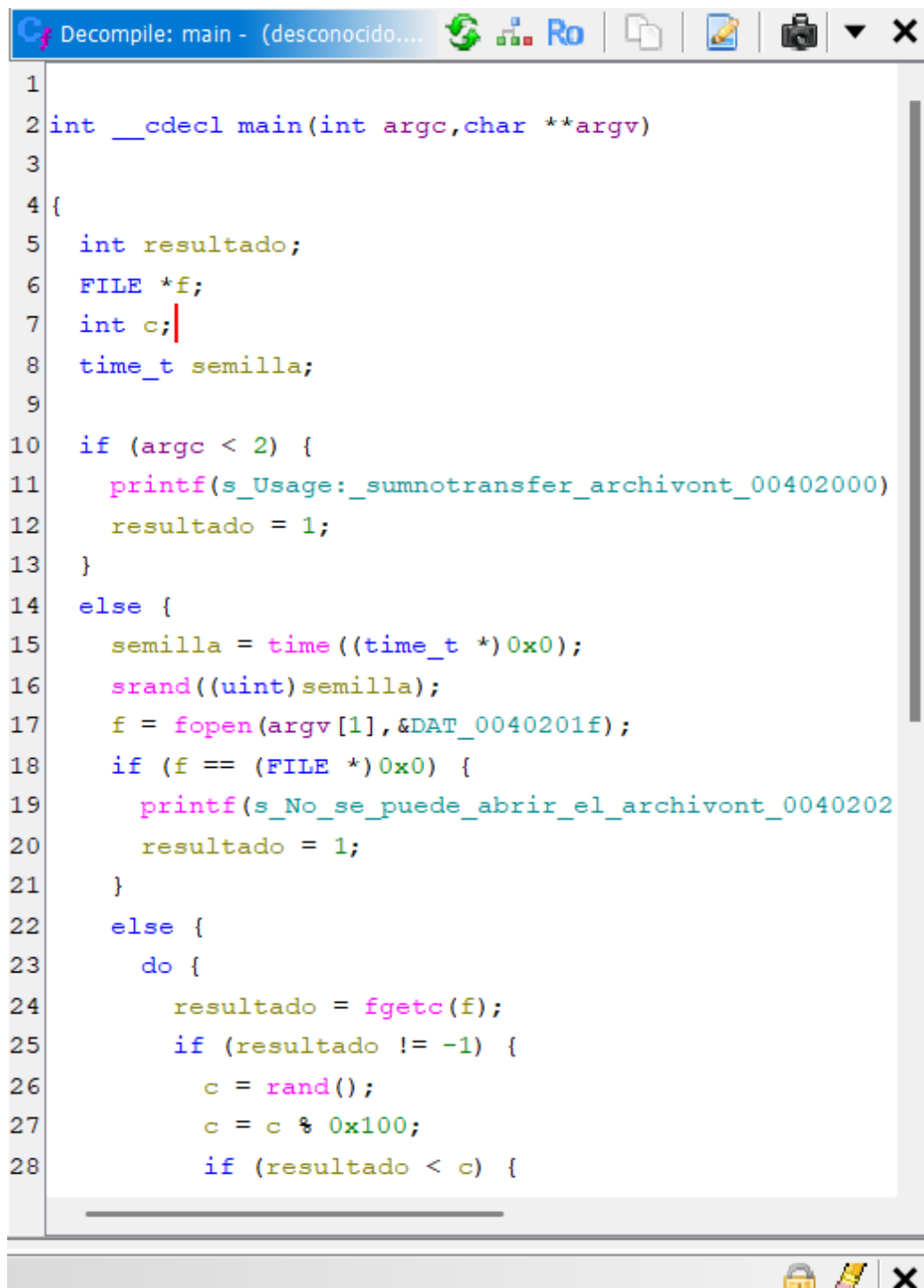
La corrección queda de esta manera:



Al aceptar la corrección, el pseudocódigo se volvió considerablemente más legible de forma automática: al reconocer argv como un arreglo de cadenas de texto, Ghidra comenzó a representar el acceso a memoria correspondiente como argv[1], en lugar de la operación aritmética sobre punteros que mostraba antes.

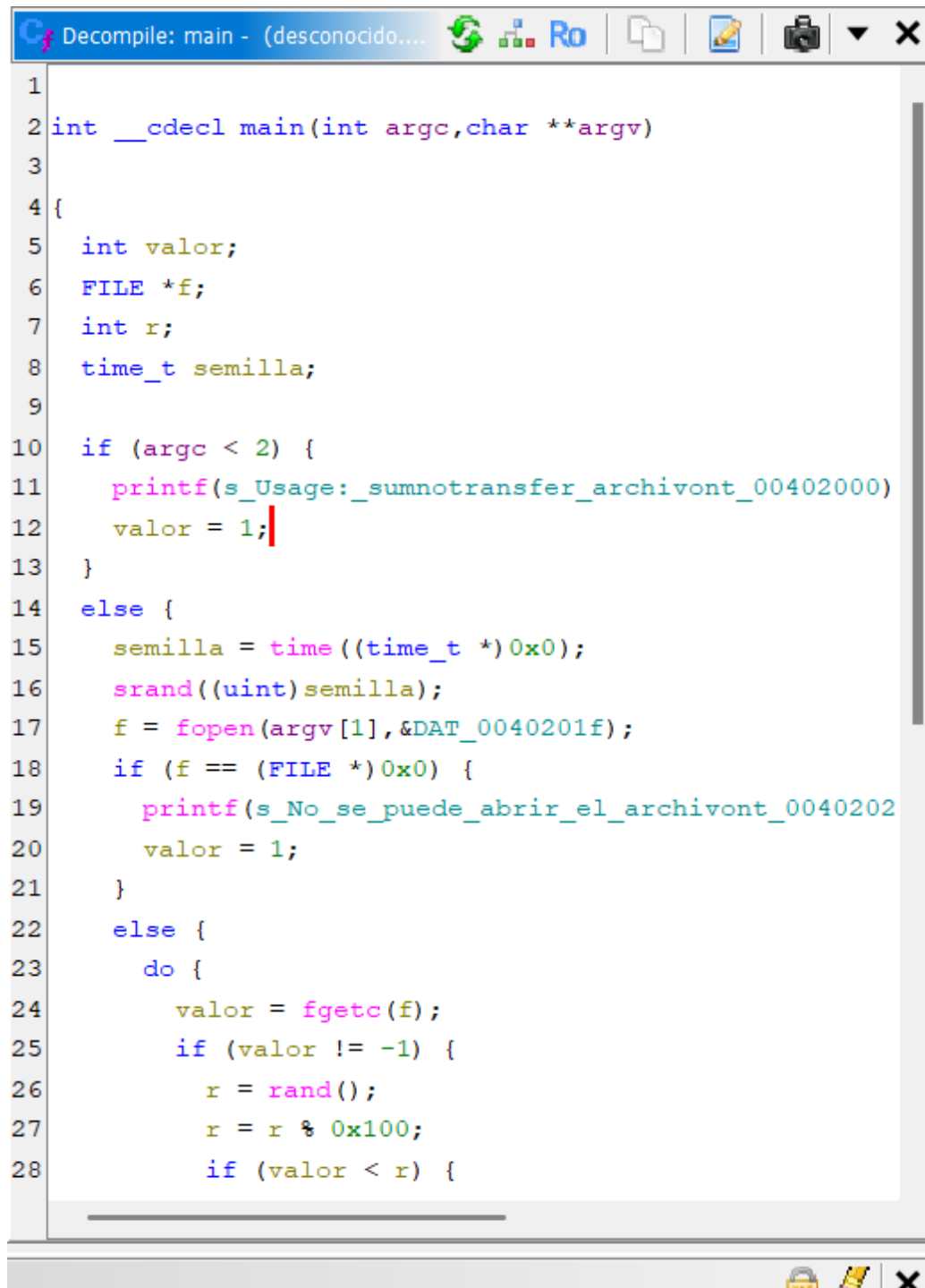
6. Identificación de las variables

El siguiente paso consistió en determinar la función de cada variable observando el contexto en que se utilizaba, para luego asignarle un nombre representativo. Por ejemplo, la variable que recibía el valor de retorno de fopen() se renombró como f; la que almacenaba el resultado de fgetc() (encargada de leer un byte del archivo) se llamó c, y la que contenía el número aleatorio generado se identificó como r.



```
1
2 int __cdecl main(int argc, char **argv)
3
4 {
5     int resultado;
6     FILE *f;
7     int c;
8     time_t semilla;
9
10    if (argc < 2) {
11        printf(s_Usage:_sumnotransfer_archivont_00402000)
12        resultado = 1;
13    }
14    else {
15        semilla = time((time_t *)0x0);
16        srand((uint)semilla);
17        f = fopen(argv[1], &DAT_0040201f);
18        if (f == (FILE *)0x0) {
19            printf(s_No_se_puede_abrir_el_archivont_0040202
20            resultado = 1;
21        }
22        else {
23            do {
24                resultado = fgetc(f);
25                if (resultado != -1) {
26                    c = rand();
27                    c = c % 0x100;
28                    if (resultado < c) {
```

Ya arreglado, quedaron separadas: valor, f, r, semilla:



```
1
2 int __cdecl main(int argc, char **argv)
3
4 {
5     int valor;
6     FILE *f;
7     int r;
8     time_t semilla;
9
10    if (argc < 2) {
11        printf(s_Usage:_sumnotransfer_archivont_00402000)
12        valor = 1;
13    }
14    else {
15        semilla = time((time_t *)0x0);
16        srand((uint)semilla);
17        f = fopen(argv[1], &DAT_0040201f);
18        if (f == (FILE *)0x0) {
19            printf(s_No_se_puede_abrir_el_archivont_0040202
20            valor = 1;
21        }
22        else {
23            do {
24                valor = fgetc(f);
25                if (valor != -1) {
26                    r = rand();
27                    r = r % 0x100;
28                    if (valor < r) {
```

Y así se ve el código completo, ya bastante entendible, con las 4 condiciones que decide el programa dependiendo de si el número aleatorio es mayor, menor o igual al byte leído:

```
Decompile: main - (desconocido.exe)

19     printf(s_No_se_puede_abrir_el_archivont_00402022,0);
20     valor = 1;
21 }
22 else {
23     do {
24         valor = fgetc(f);
25         if (valor != -1) {
26             r = rand();
27             r = r % 0x100;
28             if (valor < r) {
29                 printf(s_d-d-00402042,r,r - valor);
30             }
31             if (r < valor) {
32                 printf(s_d+d-00402049,valor - r,r);
33             }
34             if (valor == 0) {
35                 printf(&DAT_00402050);
36             }
37             if (r == valor) {
38                 printf(s_d+1_00402055,valor + -1);
39             }
40         }
41     } while (valor != -1);
42     fclose(f);
43     valor = 0;
44 }
45 }
46 return valor;
47 }
```

7. Escritura del código fuente en C

Con la lógica del programa ya comprendida, se procedió a redactar manualmente el archivo sumnotransfer.c, traduciendo el pseudocódigo generado por Ghidra a código C estándar y compilable.

```
sumnotransfer.c X
C: > Users > julie > OneDrive > Documentos > 9no semestre > Topics > Reconstruccion_Ghidra > Reconstruido >
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  int main(int argc, char **argv)
6  {
7      int resultado;
8      int c;
9      int r;
10     time_t semilla;
11     FILE *f;
12
13     if (argc < 2) {
14         printf("Usage:\nsumnotransfer archivont");
15         resultado = 1;
16     }
17     else {
18         semilla = time(0);
19         srand((unsigned int)semilla);
20         f = fopen(argv[1], "rb");
21         if (f == NULL) {
22             printf("No se puede abrir el archivont\n");
23             resultado = 1;
24         }
25         else {
26             do {
27                 c = fgetc(f);
28                 if (c != -1) {
29                     r = rand();
30                     r = r % 256;
31                     if (c < r) {
32                         printf("%d-%d\n", r, r - c);
33                     }
34                     if (r < c) {
35                         printf("%d+%d\n", c - r, r);
36                     }
37                 }
38             } while (c != -1);
39         }
40     }
41     return resultado;
42 }
```

8. Compilarlo con TCC

```
C:\Users\julie\OneDrive\Documentos\9no semestre\Topicos\Reconstruccion_Ghidra\Reconstruido>reconstruido.exe prueba.txt
10+62
226-115
151-43
135-38
189-157
121-12
223-106
203-93
221-121
145-34
104-91
254-244
```

9. Comparación funcional contra el binario original

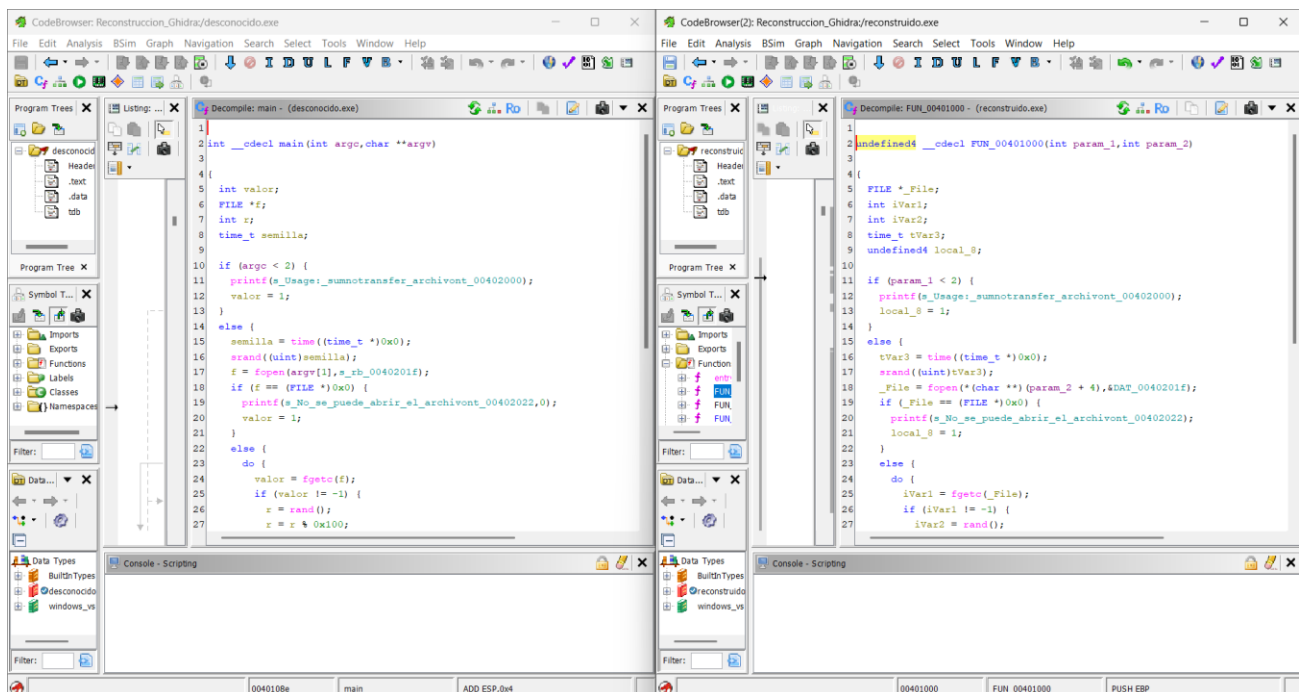
El programa utiliza números aleatorios, por lo que produce una salida distinta en cada ejecución, aunque procese el mismo archivo. Por eso no era posible comparar el texto impreso directamente entre ambos programas. Para resolverlo, se desarrolló un script en Python (decodificador.py) que interpreta las expresiones aritméticas de la salida y las resuelve, reconstruyendo el archivo original a partir de ellas. Si el decodificador se aplica tanto a la salida del original como a la del reconstruido, ambos deberían producir el mismo archivo.

```
C:\Users\julie\OneDrive\Documentos\9no semestre\Topicos\Reconstruccion_Ghidra\Reconstruido>fc /b prueba.txt rec_original.bin
Comparando archivos prueba.txt y REC_ORIGINAL.BIN
FC: no se han encontrado diferencias

C:\Users\julie\OneDrive\Documentos\9no semestre\Topicos\Reconstruccion_Ghidra\Reconstruido>fc /b prueba.txt rec_reconstruido.bin
Comparando archivos prueba.txt y REC_RECONSTRUIDO.BIN
FC: no se han encontrado diferencias
```

10. Comparación en Ghidra

Se importó también reconstruido.exe a Ghidra, siguiendo el mismo procedimiento aplicado al binario original.



Al comparar ambas ventanas líneas por línea, se observó que las dos versiones presentan exactamente la misma lógica: idéntica validación de argumentos, la misma forma de apertura del archivo, el mismo manejo de errores y el mismo bucle de lectura.